



Education

Lab 4.2. A Simple DAG Workflow

In this chapter, we begin our hands-on journey with Argo Workflow, creating simple workflows. This lab focuses on practical implementation, allowing users to familiarize themselves with Argo Workflow by deploying a workflow. This step-by-step exploration will create a minimal workflow demonstrating a DAG (**Directed-Acyclic Graph**) Workflow. DAGs are particularly useful in scenarios where tasks or steps have interdependencies, and their execution order needs to be carefully managed. DAGs can be highly beneficial in data processing pipelines, scientific workflows, machine learning, and multi-step deployment workflows. This example will help you understand how to define and run a simple DAG Workflow.

Objective

To see Argo Workflows in action:

- We will create a simple WorkflowTemplate that represents a task in our DAG Workflow, and
- A Workflow that uses the template and defines the relationship between the tasks.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
- Argo Workflows CLI (optional).

Deploy a DAG Workflow

1. Install Resources

Here is an Argo WorkflowTemplate that runs a container. Run the following command to create a manifest called `dag-workflow-template.yaml`:

```
cat <<EOF > dag-workflow-template.yaml
```

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: echo-template
  namespace: argo
spec:
  serviceAccountName: argo
  templates:
  - name: echo
    inputs:
      parameters:
      - name: message
    container:
      image: alpine:3.7
      command: [echo, "{{inputs.parameters.message}}"]
EOF
```

Create a DAG workflow that uses the template and defines the relationships between tasks. Save this workflow in a file called, **dag-workflow.yaml** as shown below:

```
cat <<EOF > dag-workflow.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: dag-diamond-
  namespace: argo
spec:
  entrypoint: diamond
  templates:
  - name: diamond
    dag:
      tasks:
      - name: A
        arguments:
          parameters: [{name: message, value: A}]
        templateRef:
          name: echo-template
          template: echo
      - name: B
        dependencies: [A]
        templateRef:
          name: echo-template
          template: echo
        arguments:
```

```
      parameters: [{name: message, value: B}]
- name: C
  dependencies: [A]
  templateRef:
    name: echo-template
    template: echo
  arguments:
    parameters: [{name: message, value: C}]
- name: D
  dependencies: [B, C]
  templateRef:
    name: echo-template
    template: echo
  arguments:
    parameters: [{name: message, value: D}]
```

EOF

To successfully deploy this Workflow we need to temporarily grant admin permissions to argo ServiceAccount with the following command:

```
$ kubectl create rolebinding default-admin --clusterrole=admin
--serviceaccount=argo:default -n argo
```

For a production environment, please follow the [official documentation](#).

As the next step, create the WorkflowTemplate shown above, and run the following command:

```
kubectl apply -f dag-workflow-template.yaml
```

Then use the following `kubectl create` command to submit the DAG workflow to your Kubernetes cluster with a generated name:

```
kubectl create -f dag-workflow.yaml
```

2. Inspect Workflow Resource

Check whether the WorkflowTemplate and Workflow resources have been created.

Command:

```
$ kubectl -n argo get workflowtemplates.argoproj.io
```

Output:

NAME	AGE
echo-template	13m

Command:

```
$ kubectl -n argo get workflow
```

Output:

NAME	STATUS	AGE	MESSAGE
dag-diamond-fnct6	Running	13s	

You can achieve the same by using the Argo Workflow CLI as follows:

```
$ argo -n argo list
```

Output:

NAME	STATUS	AGE	DURATION	PRIORITY	MESSAGE
dag-diamond-fnct6	Succeeded	34s	30s	0	

Now let's review the details of the Workflow that we just submitted. The output for the command below will be the same as the information shown when you submitted the Workflow:

```
$ argo -n argo get dag-diamond-fnct6 # Use the name from the previous command
```

Output:

```
Name: dag-diamond-fnct6
Namespace: argo
ServiceAccount: unset (will run with the default ServiceAccount)
Status: Succeeded
Conditions:
  PodRunning: False
  Completed: True
Created: Mon Nov 03 20:39:27 +0000 (52 seconds ago)
Started: Mon Nov 03 20:39:27 +0000 (52 seconds ago)
Finished: Mon Nov 03 20:39:57 +0000 (22 seconds ago)
Duration: 30 seconds
Progress: 4/4
ResourcesDuration: 0s*(1 cpu),16s*(100Mi memory)
```

STEP	TEMPLATE	PODNAME
✓ dag-diamond-fnct6	diamond	
└─✓ A	echo-template/echo	
dag-diamond-fnct6-echo-2681686565		4s
└─✓ B	echo-template/echo	
dag-diamond-fnct6-echo-2631353708		6s

```
|─✓ C          echo-template/echo
dag-diamond-fnct6-echo-2648131327  5s
|─✓ D          echo-template/echo
dag-diamond-fnct6-echo-2597798470  4s
```

We can also check the logs of the Workflow with the following command:

```
$ argo -n argo logs dag-diamond-fnct6
```

Output:

```
dag-diamond-fnct6-echo-2681686565: A

dag-diamond-fnct6-echo-2681686565: time="2025-11-03T20:39:30.999Z"
level=info msg="sub-process exited" argo=true error="<nil>"

dag-diamond-fnct6-echo-2648131327: C

dag-diamond-fnct6-echo-2648131327: time="2025-11-03T20:39:41.585Z"
level=info msg="sub-process exited" argo=true error="<nil>"

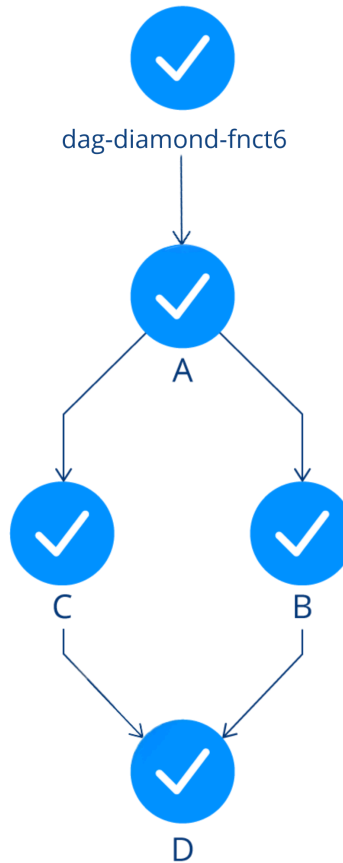
dag-diamond-fnct6-echo-2631353708: B

dag-diamond-fnct6-echo-2631353708: time="2025-11-03T20:39:42.659Z"
level=info msg="sub-process exited" argo=true error="<nil>"

dag-diamond-fnct6-echo-2597798470: D

dag-diamond-fnct6-echo-2597798470: time="2025-11-03T20:39:50.132Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

As we can see, in this workflow, step A runs first, since it has no dependencies. After A has finished, then steps B and C run in simultaneously. When B and C are completed, step D starts. You can also view the completed steps in Argo UI as shown below.



A simple DAG workflow

3. Clean Up the Resources

We successfully created an argo Workflow. To keep our working cluster nice and clean, we are going to clean up the resources we created using the commands below:

```
$ kubectl -n argo delete workflow dag-diamond-fnct6
$ kubectl -n argo delete workflowtemplates.argoproj.io echo-template
```

4. Summary

In this exercise, we successfully implemented a lab setup by deploying DAG Workflows. Within a Kubernetes environment, we demonstrated how to define dependencies between tasks which can be well-suited for use cases that involve complex dependencies and workflows. For example, data extraction must be completed before data transformation, and transformation must finish before data loading. Argo Workflow DAG allows you to model and execute these dependencies efficiently.